

ÉCOLE POLYTECHNIQUE FÉDÉRALE DE LAUSANNE

SEMESTER PROJECT SPRING 2026

Adaptive Compute Allocation for Lean Proof Generation

Author:

Oscar Johan Høeg WOHLFAHRT

Supervisor:

Etienne BAMAS



Contents

1	Introduction	1
2	Background and Preliminaries	2
2.1	Formal theorem proving with Lean	2
2.2	Proof generation with LLMs	2
2.3	Sampling, temperature, and pass@k	2
2.4	Power sampling and distribution sharpening	3
2.5	Scalable power sampling	4
2.6	Adaptive compute allocation during proof generation	4
3	Methodology	5
3.1	Task setup and verification	5
3.2	Baseline method	5
3.3	Metropolis–Hastings power sampling	5
3.4	Scalable power sampling	6
3.5	Adaptive budget controllers	6
3.5.1	Phase-based heuristic controllers	6
3.5.2	Learned XGBoost controller	7
3.6	Feedback-based extension	8
3.7	Uncertainty diagnostics	8
4	Experimental Setup	9
4.1	Models	9
4.2	Benchmarks	9
4.3	Metrics	9
4.4	Experimental settings	9
4.4.1	Baseline sampling	10
4.4.2	Metropolis–Hastings power sampling	10
4.4.3	Scalable power sampling	10
4.4.4	Adaptive controllers	10
4.4.5	Feedback experiments	11
4.5	Uncertainty experiments	11
5	Results	11
5.1	Baselines	11
5.2	Power sampling	12
5.2.1	Metropolis Hastings power sampling	12
5.2.2	Scalable Power Sampling	12
5.3	Adaptive controllers	14
5.3.1	Phased controllers	14
5.3.2	Learned Controllers	14
5.4	Additional experiments	15
5.4.1	Feedback extension	15
5.4.2	Uncertainty analysis	15
5.4.3	Results on other benchmarks	16
6	Discussion	17
6.1	Effect of distribution sharpening	17
6.2	Accuracy–cost trade-offs at pass@1 and pass@k	17
6.3	Takeaways from adaptive controllers	18
6.4	Uncertainty as a signal for proof generation	18
6.5	Limitations	19
7	Conclusion	19
7.1	Future Work	20
	Acknowledgements	21

Statement on the use of AI tools	21
Appendix A Pseudocode for Scalable Power Sampling	23
Appendix B Pseudocode for Metropolis–Hastings Power Sampling	24

1 Introduction

Large language models have recently had a significant improvement in mathematical reasoning abilities, in both formal and informal theorem proving [8], [13]. In formal theorem proving, the goal is not only to produce a correct mathematical argument, but also to generate a proof that can be checked by a proof assistant like Lean. This makes the setting especially interesting for evaluating reasoning systems, since the generated proof can be accepted or rejected automatically by the verifier.

Generating correct formal proofs is however still challenging, even with the clean verification signal. For a proof to be accepted, the model must generate Lean code that is syntactically correct, compatible with the verifier environment, and closes all proof goals. A small local mistake can cause the entire proof to fail, and the search space of possible proofs is very large. The success rate can depend heavily on how inference-time compute is used [14]. There are many possible ways to allocate this compute, for instance one can sample more proofs [3], use more expensive search procedures, or apply methods that sharpen the model distribution toward more likely reasoning trajectories [10].

Recent work has suggested that part of the benefit of reinforcement learning post-training may come from sharpening a model’s existing output distribution rather than teaching it entirely new capabilities, [16]. This has motivated training-free alternatives to RL, such as power sampling, where the goal is to sample from a sharpened distribution over full generated sequences [10]. Exact sampling from this sequence-level distribution is challenging in the autoregressive setting, as the probability of the next token under the sharpened distribution does not only depend on its local probability, but on the total probability mass of all possible future continuations after that token. Computing this exactly would require summing over an exponentially large space of continuations, so practical methods must approximate the distribution using additional inference-time compute. Newer scalable variants attempt to make this more efficient by approximating the effect of trajectory-level sharpening during autoregressive generation [9].

In this project, we study these ideas in the setting of Lean proof generation. Our main question is whether inference-time compute can be allocated more effectively when generating formal proofs. Standard fixed-budget methods spend the same amount of compute throughout generation, even though different parts of a proof may have different levels of difficulty or importance. We therefore investigate both fixed-budget sampling methods and adaptive budget controllers that choose how much compute to spend at different stages of the generation.

Throughout the project we evaluate several approaches. First, we compare standard sampling baselines with power-sampling methods under different compute budgets. Second, we study simple adaptive controllers based on proof position. Third, we train a lightweight learned controller on generated proof trajectories, using proof-position and uncertainty features to choose between sampling modes. The goal is to improve the trade-off between proof success and compute cost. Throughout the report, generated proofs are checked using Lean, and performance is measured by proof success rate and inference-time compute usage.

Our experiments show that additional inference-time compute can improve single-attempt proof generation, but that the accuracy gains come at a highly increased token cost. The advantage does however become less clear when compared with baselines using multiple independent samples. The adaptive controllers explored in this project do not consistently improve over the best fixed-budget configurations. The methods highlight some of the practical difficulties of adaptive compute allocation, for instance, that the available sampling modes must differ meaningfully in their accuracy–cost behaviour, and that learned controllers require informative features, training trajectories, and verification labels.

Project overview The report makes the following main contributions:

- We evaluate standard sampling baselines and power-sampling methods for Lean proof generation under different compute budgets.
- We compare the accuracy–cost trade-off of single-attempt rollout-based sampling with independent pass@ k sampling.
- We investigate simple phase-based controllers and a lightweight learned controller for adaptive inference-time compute allocation.

- We perform exploratory uncertainty diagnostics to study whether model likelihoods provide a useful signal for proof failure and adaptive control.

2 Background and Preliminaries

Before presenting the methods and experimental results, we will introduce the background knowledge needed to understand the project. We first give a brief overview of formal theorem proving with Lean, and describe autoregressive proof generation with language models, standard sampling methods, and pass@ k evaluation. Finally, we introduce power sampling, scalable power sampling, and the idea of adaptive allocation of inference-time compute.

2.1 Formal theorem proving with Lean

Lean is an interactive theorem prover and programming language used for formal mathematics [5]. In Lean, mathematical proofs are written as formal theorem statements, and a proof is accepted only if it passes Lean’s verifier. This contrasts with natural language mathematics, where a proof only needs to convince the reader.

In this project, we use Lean as an automatic verifier for the generated proofs. The model is given a formal theorem statement and must generate a proof continuation. The generated text is then inserted into Lean and checked. If Lean accepts the proof, the attempt is counted as successful, otherwise, it is counted as a failure. This makes it possible to automate the verification of the proof attempts, and thus feasible to compare different inference strategies.

Lean proof generation is still however a difficult task. A proof attempt can fail for many reasons, for instance, syntax errors, wrong theorem names, invalid tactics, missing intermediate steps, or proof steps that are locally plausible but does not actually close the current goal. The model has to generate code that is not only valid Lean, but also a complete proof of the given theorem.

2.2 Proof generation with LLMs

Large language models generate text autoregressively, which means that they define a probability distribution over the next token conditioned on the prompt and the tokens generated so far [2]. Given an input prompt q , which in our case consists of the Lean theorem statement and the necessary context, the model defines a conditional distribution over possible proof continuations. If we write the generated proof as a sequence of tokens $x_{1:T} = (x_1, \dots, x_T)$, then the probability assigned to the full sequence can be written as

$$p(x_{1:T} | q) = \prod_{t=1}^T p(x_t | q, x_{<t}),$$

where $x_{<t}$ denotes the tokens generated before step t . A proof is therefore sampled by repeatedly sampling the next token from the model’s conditional distribution.

This autoregressive structure is important for Lean proof generation. At each step, there may be many locally plausible token choices, but a single invalid or poorly chosen step can make the entire proof attempt fail. An early mistake can also lead the generation into a proof state where the original theorem can no longer be solved. Some correct proofs may also require tactics, lemmas, or intermediate steps that are locally unlikely, but necessary for completing the proof. Thus, the most likely local continuation is not always the one that leads to a valid proof.

This motivates the study of inference-time sampling methods. Different decoding strategies change how the next token is chosen from $p(x_t | q, x_{<t})$, and therefore change the distribution of complete proof attempts. In the following sections, we describe several such strategies, starting from standard temperature-based sampling and then moving to power-sampling-inspired methods that aim to sharpen the distribution over full proof attempts.

2.3 Sampling, temperature, and pass@ k

The autoregressive model defines a probability distribution over the next token, but we still need to decide on how tokens are selected during generation. The simplest option is greedy decoding, where we choose the most likely token at each step. This can however get stuck in very similar proof attempts [7]. A nice alternative is

therefore to sample from the model distribution, which makes it possible to generate several different proof candidates for the same theorem. A common way to achieve this is using temperature sampling. Given a next-token distribution $p(x_t | q, x_{<t})$, temperature sampling is defined as,

$$p_\tau(x_t | q, x_{<t}) = \frac{p(x_t | q, x_{<t})^{1/\tau}}{\sum_{x' \in \mathcal{V}} p(x' | q, x_{<t})^{1/\tau}},$$

where $\tau > 0$ denotes the temperature. Lower τ makes the distribution sharper and makes high-probability tokens more likely, while higher τ makes the distribution flatter and increases diversity. This creates a trade-off, as a sharper distribution may avoid unlikely and syntactically invalid continuations, but also reduces the amount of exploration. A flatter distribution may find proofs that greedy or low-temperature decoding misses, but could also produce more invalid attempts.

A simple way to improve success rate is to generate several independent candidates and accept the theorem if at least one of them passes. This is referred to as pass@ k , which is the fraction of problems where at least one of k attempts is verified [3]. Different sampling strategies can behave differently as k increases. Sharper methods may perform better at pass@1 because they make high-likelihood completions more likely, whereas less sharp methods may perform better at larger k , by producing more diverse candidates. Therefore, it is important to consider the number of generated candidates and the token cost required to produce them, in addition to the success rate of a method.

In the rest of the report, we therefore evaluate methods by both proof success and inference-time compute. This makes it possible to compare strategies that use different amounts of sampling, sharpening, or search, and motivates the later question of whether this compute should be allocated uniformly or adaptively during proof generation.

2.4 Power sampling and distribution sharpening

The temperature based sampling method described above modifies the distribution of the model locally. A different possible approach is to sharpen the distribution over full generated sequences. This perspective is motivated by recent work suggesting that some of the gains in reasoning from reinforcement learning post-training may come from concentrating probability mass on better reasoning trajectories that are already present in the base model, rather than unlocking entirely new capabilities [16].

We again let q denote the input prompt, $x_{1:T}$ be a complete generated proof, and define the base model distribution as $p(x_{1:T} | q)$. Following Karan and Du [10], power sampling defines a sharpened sequence-level distribution,

$$p_\alpha^{\text{pow}}(x_{1:T} | q) = \frac{p(x_{1:T} | q)^\alpha}{\sum_{\tilde{x}_{1:T}} p(\tilde{x}_{1:T} | q)^\alpha}, \quad \alpha > 1.$$

Increasing α concentrates more probability mass on high-likelihood sequences. Importantly, this sharpening is applied to the probability of the entire generated trajectory, not only to the next-token.

This makes power sampling different from low-temperature sampling. Low-temperature sampling sharpens the distribution over the next token given the current prefix. Power sampling instead sharpens the distribution over complete continuations. To see this, consider the next-token distribution induced by the sequence-level power distribution. Here $x_{>t}$ denotes the future suffix after the fixed next token x_t , while $\tilde{x}_{t:T}$ denotes an arbitrary continuation from position t onward. We then have

$$p_\alpha^{\text{pow}}(x_t | q, x_{<t}) = \frac{\sum_{x_{>t}} p(x_{<t}, x_t, x_{>t} | q)^\alpha}{\sum_{\tilde{x}_{t:T}} p(x_{<t}, \tilde{x}_{t:T} | q)^\alpha}.$$

This expression shows that the probability of choosing a token x_t does not only depend on its local probability, but also on the probability of the future completions that follow from it. In other words, power sampling can prefer a token that may not be the most likely local choice, but that leads to better complete trajectories. This is especially interesting in theorem proving, as we might sometimes need to use tactics that seem locally unlikely, but lead to correct proofs down the line.

Exact sampling from p_α^{pow} does however come at an infeasibly large computational cost. Computing the next-token distribution would require summing over all possible future completions after each candidate token. Karan and Du propose an approximate Markov chain Monte Carlo approach that uses the base model’s own

likelihoods to sample from this sharpened distribution without additional training or access to a verifier [10]. This makes power sampling feasible, but the iterative MCMC procedure can be expensive at inference time. The next section describes a more scalable approximation, which motivates the fixed-budget and adaptive sampling methods used in our experiments.

2.5 Scalable power sampling

The main difficulty with power sampling is that the next-token distribution depends on all future continuations. This very quickly becomes intractable to compute, as the number of possible continuations grows exponentially with sequence length.

Scalable Power Sampling, proposed by Ji et al. [9], provides a more efficient approximation to the MCMC based method of Karan and Du. The key observation is that the power distribution can be written as a locally sharpened next-token distribution multiplied by a token-specific scaling factor. Informally, this gives,

$$p_{\alpha}^{\text{pow}}(x_t | q, x_{<t}) \propto p(x_t | q, x_{<t})^{\alpha} \xi_t(x_t),$$

where $p(x_t | q, x_{<t})^{\alpha}$ is the low-temperature local sharpening term. The scaling factor $\xi_t(x_t)$ accounts for the powered probability mass of future continuations after choosing x_t . In the ideal full-sequence setting, it can be written as,

$$\xi_t(x_t) = \sum_{x_{>t}} p(x_{>t} | q, x_{<t}, x_t)^{\alpha},$$

where $x_{>t}$ ranges over all possible future suffixes. Thus, $\xi_t(x_t)$ is large when the candidate token x_t leads to many high-likelihood future continuations. If this term were ignored, the method would reduce to ordinary local low-temperature sampling rather than approximating the sequence-level power distribution.

One issue does however still remain. $\xi_t(x_t)$ cannot be computed exactly, so it must be approximated. The method proposed by Ji et al. [9] does this by using the base model itself to estimate the likelihood of short future rollouts. For each candidate next token, the model generates a limited number of future continuations and scores these continuations using their likelihood under the base model. Since

$$\sum_{x_{>t}} p(x_{>t} | q, x_{\leq t})^{\alpha} = \mathbb{E}_{x_{>t} \sim p(\cdot | q, x_{\leq t})} \left[p(x_{>t} | q, x_{\leq t})^{\alpha-1} \right],$$

the future-mass term can be estimated from sampled rollouts by averaging powered rollout likelihoods. These rollout scores provide a Monte Carlo estimate of which candidate tokens have more future probability mass under the approximate power distribution. The quality of the approximation depends on parameters such as the number of candidate tokens, the number of rollout continuations, and the rollout horizon. Increasing these parameters can improve the estimate of the scaling term, but also increases token cost.

This compute–accuracy trade-off motivates the adaptive methods studied later in the report. Fixed SPS configurations use the same rollout budget throughout generation, even though some proof steps may be nearly forced while others determine which proof path the model follows. This suggests that it may be useful to choose between cheaper and more expensive sampling modes during generation, rather than using a single budget everywhere.

2.6 Adaptive compute allocation during proof generation

The methods described above all attempt to improve accuracy by spending more compute at inference time. This extra computation may come from generating multiple independent candidates, running MCMC refinement steps, or scoring using rollouts. More generally, this fits into a broader trend where language-model performance is improved not only by training larger or stronger models, but also by allocating more compute at test time [14]. The question considered in this project is whether this additional compute should be spent uniformly throughout proof generation, or whether it can be allocated adaptively.

A fixed compute budget is unlikely to be optimal in all situations. At the token level, some choices may be almost forced by the current prefix, while others may determine which proof path the generation follows. At the theorem level, some problems are solved easily with cheap sampling, while others may require more exploration or more careful candidate selection. This suggests that the useful amount of inference-time compute can vary both across proof states and across problems.

Instead of choosing one sampling configuration for the entire proof, an adaptive method chooses between several budget modes during generation. For example, the model may use a cheap mode when the next step appears confident, and a more expensive mode when the current state appears uncertain or important. The goal is not necessarily to maximize success at any cost, but to improve the trade-off between success and compute.

There are several possible signals for deciding when to spend more compute. One simple signal is the position in the proof, as early decisions may influence the rest of the trajectory, while later decisions may be closer to closing the goal. Another option is model uncertainty, measured for example through token log-probabilities, entropy, or other statistics computed during generation. Recent work has explored token-level uncertainty as a signal for unreliable reasoning steps [17]. In our setting, this motivates testing whether uncertainty is correlated with Lean verification failure, and whether such signals can help decide when to allocate a larger sampling budget.

The next section describes the concrete task setup, sampling methods, and controller implementations used in the experiments.

3 Methodology

3.1 Task setup and verification

Our task is formal to formal proof solving. For each problem, we give the model a prompt containing the needed Lean setup and a theorem to prove. The model then generates a proof continuation in Lean. After generation, we extract the Lean code block and pass the complete theorem and generated proof to the Lean verifier. We also apply a cleanup step to handle generations that end with incomplete syntax at the maximum token boundary. This step can remove trailing incomplete lines or unmatched delimiters, but it does not add new Lean code, introduce tactics, or modify the middle of the generated proof.

A proof attempt is counted as successful only if Lean accepts it. In addition to recording if the attempt is successful or not, we record the number of generated-tokens. For methods that use more candidates, rollouts or other techniques in their generation we count the tokens used in all of these steps.

3.2 Baseline method

We first evaluate ordinary autoregressive generation from the used LLM. For this method, the model generates a proof continuation directly, without using any additional sampling techniques. This gives us a cheap generation baseline which we can compare the later methods against.

For the baseline we consider both standard sampling and temperature-scaled sampling. In addition, we evaluate both pass@1 and pass@k. The specific models, sampling parameters, and numbers of generated candidates are reported in the experimental setup section.

3.3 Metropolis–Hastings power sampling

As a first power-sampling reference point, we evaluate an autoregressive Metropolis–Hastings (MH) sampler inspired by Karan and Du [10], as introduced in Section 2.4. We do not evaluate their direct full-sequence MH variant. Instead, we use the blockwise variant, where a proof is constructed progressively over multiple generation blocks.

At each block, the sampler first extends the current proof prefix with a proposal continuation. It then performs a fixed number of MH refinement steps before the resulting state is used as the prefix for the next block. In each refinement step, a cut position is sampled in the generated proof, the suffix after this cut is resampled up to the current block boundary, and the proposed state x' is accepted from the current state x with probability

$$A(x' | x) = \min\left(1, \frac{p(x')^\alpha q(x | x')}{p(x)^\alpha q(x' | x)}\right),$$

where p is the base model distribution, q is the proposal distribution, and α controls the strength of the sharpening. The procedure continues until the model emits a proof-ending stop sequence or the maximum token budget is reached.

For Lean, we make two adaptations. Firstly, we apply the same deterministic Lean extraction procedure used by our baseline prover before verification. Secondly, we restrict the allowed cut positions to be after the problem statement and theorem header. This ensures that the definition of the theorem we intend to prove is preserved, but still allows for sampling inside the proof body.

3.4 Scalable power sampling

The Metropolis–Hastings method described above gives a direct way to approximately sample from the power distribution. However, it is expensive, as it requires iteratively refining the generated continuations using MCMC updates. We therefore also evaluate a Lean-adapted version of Scalable Power Sampling, proposed by Ji et al. [9], as introduced in 2.5. This method also constructs the proof step by step, but it avoids running an MCMC chain over candidate continuations. Instead, Scalable Power Sampling approximates the power distribution by combining local token probabilities with rollout-based estimates of the quality of future continuations.

The original Scalable Power Sampling method is defined at the token level, but Ji et al. also describe a batched version where the same idea is applied to short blocks of tokens. Our Lean implementation follows this block-level variant. Starting from the current proof prefix, the model first proposes several candidate next chunks. A subset of these candidates is then evaluated using short future rollouts. Each rollout is scored using the base model likelihood, and the candidate likelihoods and rollout scores are combined into a power-sampling weight that approximates the local sharpening term and the future-dependent scaling term described in Section 2.5. The selected chunk is appended to the proof prefix, and the procedure is repeated until the model emits a stop sequence or the maximum token budget is reached.

This method introduces several tunable parameters: the number of candidate chunks, the number of candidates evaluated with rollouts, the number of rollouts per candidate, the rollout horizon, the chunk size, and the power exponent α . By varying these parameters, we obtain sampling configurations with different accuracy–cost trade-offs. These configurations are later used both as fixed-budget baselines and as the action space for adaptive controllers, where the goal is to choose between cheaper and more expensive sampling modes during proof generation.

3.5 Adaptive budget controllers

The fixed sampling methods described above use a single configuration throughout proof generation. We also evaluate adaptive controllers that choose between predefined sampling modes during generation. The motivation is that some proof states may be relatively mechanical, while others may be more uncertain or more important for the final proof. The goal is therefore to improve the trade-off between verification success and generated-token cost by allocating additional compute where it is the most useful.

We consider two controller types. First, we test phase-based heuristic controllers, where the sampling budget depends only on the current position in the proof. Second, we train a lightweight learned controller using XGBoost, which uses proof-position, theorem, and uncertainty-related features to choose between budget modes. The uncertainty features are motivated by the uncertainty analysis in Section 3.7, where we investigate whether model likelihoods contain useful information about proof failure.

3.5.1 Phase-based heuristic controllers

We first consider simple heuristic controllers based on the current phase of the proof. The motivation is that different parts of a proof may benefit from different sampling budgets. In particular, early proof choices can strongly constrain the remainder of the proof, so it may be useful to allocate more sampling compute shortly after the proof begins. Later parts of the proof are conditioned on a longer prefix and may be more constrained, which might make cheaper configurations sufficient.

The phase is measured by the fraction of the proof-body token budget that has already been consumed. We implement several phase-based controllers that use a more expensive sampling mode at the beginning of the proof body, an intermediate mode in the middle, and a cheaper mode near the end. The variants differ in where these phase transitions occur.

3.5.2 Learned XGBoost controller

Finally, we implement a lightweight learned controller using XGBoost [4]. The goal is to learn a policy over the fixed sampling modes. At inference time the controller is given the current generation state, and chooses which sampling mode should be used for the next part of the proof.

The controller is trained from proof-generation traces collected with a randomized behavior policy. Each proof attempt is decomposed into several state-action training rows, one for each sampling step. For each state-action row, we record the state at that sampling step, the action taken, the final Lean outcome of the complete proof attempt, and an estimate of the sampler compute used from that step until the generated attempt terminated. The action space is represented by three mode labels, `vanilla`, `cheap`, and `expensive`, crossed with $\alpha \in \{2, 4\}$. The `vanilla` mode does not use power sampling, so its two α modes are equivalent, resulting in five modes in total.

The controller features are chosen to capture the current proof state, the recent choices of the method, and properties of the theorem. Progress features describe how far we are in the generation, such as the remaining budget and the number of generated chunks. Controller-history features record recent decisions, including the previously used value of α . Sampling features are computed from the normalized candidate weights after combining the local likelihood term with the estimate of the future scaling term. These include moving averages of the effective-sample-size ratio and normalized sampling-weight entropy, the recent change in effective-sample-size ratio, and the variance of candidate log-probabilities at the previous step. Together, these diagnostics may indicate whether the sampler strongly preferred one candidate continuation, or if the candidate weights were more even. Finally, static theorem features describe the input theorem itself, such as its length and counts of equality, implication, and quantifier symbols.

The uncertainty-related and sampling-diagnostic features are included because model likelihoods may contain information about fragile or error-prone parts of a proof attempt. We investigate this separately in Section 3.7. These diagnostics should therefore be viewed both as controller features and as motivation for the uncertainty analysis.

We train two XGBoost models on these state-action rows. The first is a success classifier,

$$\widehat{P}(\text{success} \mid s_t, a),$$

which predicts the probability that the full proof attempt will be accepted by Lean for a given state-action pair. The second is a cost-to-go regressor,

$$\widehat{C}_{\text{to-go}}(s_t, a),$$

which predicts the remaining generated-token cost from the current decision point. Although the immediate cost of each sampling mode is approximately known, the total cost-to-go is state-dependent. It depends on the remaining token budget, whether the generation is close to termination, how many future controller decisions are likely, and whether the selected continuation causes early stopping.

The two models are trained independently. The success model is trained with binary logistic loss, using the final Lean outcome of the full proof attempt as the label for each in-proof decision row:

$$\mathcal{L}_{\text{succ}} = - \sum_i w_i [y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i)].$$

Here $y_i \in \{0, 1\}$ is the final Lean outcome of the proof attempt associated with row i , with $y_i = 1$ if the proof is accepted by Lean and $y_i = 0$ otherwise. The prediction $\hat{p}_i = \widehat{P}(\text{success} \mid s_i, a_i)$ is the classifier’s predicted success probability for the state-action pair, and w_i is the sample weight for that row.

The cost model is trained with a squared-error loss against the observed cost-to-go:

$$\mathcal{L}_{\text{cost}} = \sum_i w_i (\hat{c}_i - c_i)^2.$$

Here c_i is the observed remaining generated-token cost from row i , and $\hat{c}_i = \widehat{C}_{\text{to-go}}(s_i, a_i)$ is the predicted cost-to-go.

At inference time, the controller evaluates every candidate action and combines the success and cost predictions using the utility

$$U(s_t, a) = \widehat{P}(\text{success} \mid s_t, a) - \lambda \frac{\widehat{C}_{\text{to-go}}(s_t, a)}{T},$$

where T is the maximum generation token budget and is used to normalize the predicted cost. The parameter λ controls the trade-off between verification success and generated-token cost. The controller selects the action with the highest utility.

Unlike the methods described earlier, this controller is of course not training-free. It requires generated proof trajectories and Lean verification in order to train the success and cost models. We therefore see it as a lightweight learned policy for studying adaptive compute allocation, rather than as a replacement for verifier-free sampling methods. In particular, it provides a way to test whether the available state features contain enough information to choose between sampling budgets more effectively than a fixed configuration.

3.6 Feedback-based extension

As an additional extension, we evaluate the sampling methods on a proof-generation model which produces feedback blocks during the generation process, as suggested by our supervisor. This model is fine-tuned to generate feedback text during the proof process, for example after tactic blocks. The goal of this experiment is to test whether the power-sampling methods behave differently when the model is trained not only to generate Lean proof text, but also to produce intermediate feedback outputs.

We compare this feedback model against the corresponding non-feedback DeepSeek-Prover model. First, we evaluate the feedback model with ordinary generation, giving a baseline for how it performs without additional sampling methods. We then apply Scalable Power Sampling, to test whether distribution sharpening improves the proof-generation process when feedback text is present.

3.7 Uncertainty diagnostics

In addition to evaluating different sampling methods, we also perform a small number of exploratory experiments on whether model uncertainty can be used as a signal for proof errors. This is motivated by recent work studying token-level uncertainty in language-model reasoning, where likelihood-based signals are used to detect unreliable parts of a generated solution [17]. In our setting, Lean gives us an external verifier, which lets us compare the uncertainty with verified failure locations.

For a generated token x_i , we define token uncertainty as the negative log-likelihood under the model,

$$u_i = -\log p_\theta(x_i \mid x_{<i}).$$

High values of u_i correspond to tokens that the model assigned low probability during generation. For a span of tokens S , we consider three aggregate uncertainty measures,

$$U_{\text{total}}(S) = \sum_{i \in S} u_i, \quad U_{\text{mean}}(S) = \frac{1}{|S|} \sum_{i \in S} u_i, \quad U_{\text{max}}(S) = \max_{i \in S} u_i.$$

The total NLL is length-dependent, so it is mainly useful when comparing candidates of similar length. The mean and maximum token NLL are less directly affected by proof length.

These experiments are intended only as simple diagnostics. A model can of course be confidently wrong, and some correct proofs may require low-probability tokens, such as uncommon lemmas, theorems, or unusual tactics. The goal is therefore not to treat uncertainty as a direct correctness measure, but to test whether a model’s uncertainty provides a useful signal for prioritizing candidates or selecting locations for repair.

Localizing errors In the first set of experiments, we investigate whether high-uncertainty tokens occur near the first error reported by Lean. For each proof attempt, we extract the generated Lean code and send it to the Lean verifier. Accepted candidates are excluded from this experiment. For rejected candidates, we take the first Lean error message and map its line and column number back to a character offset in the generated Lean code. We then align this character offset with the generated token sequence using tokenizer offset mappings. The analysis is restricted to the proof body, excluding imports and the theorem statement.

We evaluate two quantities. First, we rank all proof-body tokens by NLL and record the rank percentile of the token aligned with the first reported Lean error. This measures whether the reported error occurs at an unusually uncertain token. Second, we measure the normalized distance between the token aligned with the first Lean error and the highest-NLL token in the proof body. Negative distances indicate that the highest-uncertainty token occurs before the first reported Lean error.

Classifying proof candidates In the second experiment, we investigate whether uncertainty over an entire proof attempt can predict whether the attempt passes Lean verification. For each theorem, we generate one proof candidate, verify it with Lean, and compute uncertainty statistics over the full generated completion. We then compare these metrics with the Lean outcome. Our main metric is the area under the ROC curve (AUC), which is a standard ranking metric for binary classification [6]. An AUC of 0.5 corresponds to random ranking, while an AUC above 0.5 indicates that successful candidates tend to have lower uncertainty than failed candidates.

4 Experimental Setup

4.1 Models

We use two main models for proof generation in the experiments. The first is `formalmathatpfl/Apertus-8B-finetuned`, a fine-tuned version of Apertus 8B. Apertus is a fully open language-model developed by EPFL, ETH Zurich, and CSCS, and is released in 8B and 70B parameter versions [12].

The second model is `formalmathatpfl/deepseek-prover-v2-cpt-sft-1e`, a fine-tuned DeepSeek-Prover-V2 model. DeepSeek-Prover-V2 is an open-source model designed for formal theorem proving in Lean 4 [13]. We include this checkpoint as a second comparison model, and as a non-feedback reference for the later feedback experiments.

For the feedback extension, we use `formalmathatpfl/deepseek-prover-v2-cpt-sft-feedback-1e`, a related DeepSeek-Prover-V2 model fine-tuned on Lean data with feedback. This model is used only in the feedback-extension experiments, where we investigate whether sampling methods behave differently when the model generates outputs with feedback in addition to proof text.

4.2 Benchmarks

The main benchmark used in this project is MiniF2F [18]. MiniF2F is a standard benchmark for formal theorem proving, containing olympiad-style, high-school, and undergraduate-level problems. Experimentation and method development was performed on the 243 problem validation split, while final results are reported on the held-out 243 problem test split.

We also run smaller exploratory evaluations on more difficult benchmarks, including PutnamBench [15], ProofNet [1], and MathOlympiadBench, released together with Goedel-Prover-V2 [11]. These benchmarks are substantially harder for our current setup than MiniF2F, and initial accuracies were very low. We therefore report these results only as exploratory checks, rather than as full benchmark comparisons.

4.3 Metrics

Our main evaluation metric is proof success. For a single generated candidate, we report accuracy, defined as the fraction of problems for which the generated proof is accepted by Lean. When several independent candidates are generated, we report `pass@k`, where a problem is counted as solved if at least one of the k candidates is accepted by Lean.

Many of the methods attempt to improve success by spending additional inference-time compute, so we also report generated-token cost. For methods such as MCMC power sampling and Scalable Power Sampling, generated-token cost includes all internal proposal generations, candidate chunks, and rollout continuations, not only the final proof passed to Lean. This is important because these methods can produce a single final proof attempt while still using many additional model generations internally.

All main results are reported as the mean over multiple random seeds, together with the standard deviation across seeds.

4.4 Experimental settings

In this subsection we give the parameters used for the different experimental runs. Unless otherwise stated, all experiments use the same prompt template, Lean extraction and verification procedure, and maximum generation length of 2048 tokens. Method-specific parameters, such as temperature, power exponent, rollout budget, and controller mode, are described separately below.

4.4.1 Baseline sampling

For the baseline sampling experiments, we evaluate generation with different temperatures and different numbers of generated candidates. We use temperatures $\tau \in \{1.0, 0.5, 0.25\}$, and report $\text{pass}@k$ for $k \in \{1, 4, 8, 16\}$.

4.4.2 Metropolis–Hastings power sampling

The Metropolis–Hastings power-sampling experiment is included only as an exploratory feasibility check, not as a full benchmark comparison. Due to the long runtime observed during validation, we did not run a full grid of MCMC configurations on the test set. Instead, we report a single validation-set run to illustrate the cost of the method in our setup.

The run uses $\alpha = 2$, 4 MH refinement steps, and a block size of 32 tokens. This block size is comparable to the chunk size used in our Scalable Power Sampling and controller experiments, making the result useful as an implementation-level cost reference. However, since the experiment is based on two seeds and one validation configuration, it should not be interpreted as a definitive evaluation of MCMC power sampling. Its main purpose is to motivate why the rest of the report focuses on Scalable Power Sampling and adaptive controllers.

4.4.3 Scalable power sampling

Before evaluating the adaptive controllers, we evaluate a set of fixed configurations that later form the controller action space. These fixed configurations serve both as standalone baselines and as reference points for the adaptive experiments. We compare three modes: a vanilla baseline, a cheap power-sampling mode, and a more expensive power-sampling mode.

The vanilla baseline corresponds to generation without rollout-based candidate evaluation. It uses batch size 32, group size 1, top- k value 1, rollout budget 0, and horizon 1. The cheap mode uses a larger candidate batch with limited lookahead, using batch size 32, group size 4, top- k value 2, rollout budget 1, and horizon 32. The expensive mode allocates more computation to candidate evaluation, using batch size 24, group size 6, top- k value 3, rollout budget 4, and horizon 64.

For each power-sampling configuration, we sweep $\alpha \in \{1, 2, 4\}$ and report $\text{pass}@k$ for $k \in \{1, 4\}$. The case $\alpha = 1$ is included mainly as a control, since it removes the additional power sharpening while still using the same candidate-generation and selection procedure. The selected settings are based on empirical tuning on the validation set, with the goal of obtaining modes that differ meaningfully in computational cost while remaining competitive.

4.4.4 Adaptive controllers

Phase-based heuristic controllers For the first adaptive controller experiment, we evaluate the phase-based controllers introduced in Section 3.5.1. All variants use the same ordering of sampling budgets within the proof: an expensive mode at the beginning, a cheap mode in the middle, and a vanilla mode near the end. The expensive, cheap, and vanilla modes use the same budget configurations as the corresponding fixed baselines described above.

The phase schedules were selected based on tuning on the validation-set. The first variant uses the expensive mode for the first 5% of the proof-body token budget, then switches to the cheap mode, and uses the vanilla mode after 75% of the budget. The second variant also uses the expensive mode for the first 5%, but switches to the vanilla mode earlier, after 60% of the proof-body token budget. The third variant uses the expensive mode for the first 10% and switches to the vanilla mode after 50%. For each schedule, we evaluate $\alpha = 2$ and report $\text{pass}@k$ for $k \in \{1, 4\}$.

Learned controller We evaluate the XGBoost controller trained on randomized controller trajectories collected from the validation set. The training data contains 183,249 state-action rows, generated from the 243 validation problems across three random seeds. For the final test-set experiments, we sweep the cost-weight parameter $\lambda \in \{0, 0.005, 0.05\}$, which controls the trade-off between predicted proof success and predicted token cost. We report $\text{pass}@k$ for $k \in \{1, 4\}$.

4.4.5 Feedback experiments

For the feedback-extension experiments, we evaluate two pass@1 configurations using the feedback model introduced above. The first uses ordinary feedback generation, while the second applies Scalable Power Sampling throughout the proof. The power-sampling run uses the cheap budget configuration with $\alpha = 2$. Both configurations are evaluated over three seeds.

Because feedback generations include intermediate feedback blocks and therefore produce substantially longer completions than the no-feedback models, we use a larger maximum generation budget of 16,384 tokens per theorem. These results are therefore not directly comparable to the no-feedback DeepSeek-Prover-V2 experiments, but are included as an additional comparison.

4.5 Uncertainty experiments

For the uncertainty experiments, we use the Apertus model on MiniF2F. The token-localization analysis is run on generated MiniF2F candidates and is restricted to failed attempts for which both a Lean error location and tokenizer alignment are available. The candidate-level analysis is computed over verified candidates, since these have reliable Lean pass/fail labels.

5 Results

5.1 Baselines

We first evaluate standard independent sampling baselines on the MiniF2F test set. For each model, temperature, and candidate budget k , we generate k independent proof attempts per theorem and report pass@ k . Tables 1 and 2 show accuracy and generated-token cost.

We can see several trends in the results. First, increasing k consistently improves performance for both models. The largest improvement usually comes from moving from pass@1 to pass@4, while the marginal gain from pass@4 to pass@16 is much smaller. This indicates that additional independent samples are useful, but the returns are diminishing.

Second, the best temperature depends on the number of generated candidates. At pass@1, lower temperatures perform best for both models: Apertus reaches 38.68% at $\tau = 0.25$, while DeepSeek reaches 37.61% at $\tau = 0.25$. At larger k , higher temperatures become more competitive. For Apertus, $\tau = 1.0$ gives the best pass@4, pass@8, and pass@16 results, while for DeepSeek the best high- k results are obtained at $\tau = 0.5$. This suggests that once multiple candidates are sampled, additional exploration becomes more beneficial.

The token costs in Table 2 scale approximately linearly with k , as expected for independent sampling. Lower temperatures also seem to produce slightly longer proofs, for both models $\tau = 0.25$ produces the longest candidates.

Temperature τ	Pass@1 (%)	Pass@4 (%)	Pass@8 (%)	Pass@16 (%)
Apertus-8B-finetuned				
1.00	34.49 $\pm$ 1.44	45.51 $\pm$ 0.55	46.91 $\pm$ 0.65	48.48 $\pm$ 0.61
0.50	37.86 $\pm$ 1.16	44.61 $\pm$ 0.99	46.50 $\pm$ 0.77	48.23 $\pm$ 1.21
0.25	38.68 $\pm$ 0.41	43.87 $\pm$ 0.80	44.61 $\pm$ 0.37	46.01 $\pm$ 0.54
DeepSeek-Prover-v2-cpt-sft-1e				
1.00	33.50 $\pm$ 2.45	47.41 $\pm$ 1.14	51.77 $\pm$ 1.28	55.06 $\pm$ 1.87
0.50	36.46 $\pm$ 1.50	49.79 $\pm$ 1.05	52.59 $\pm$ 1.52	55.72 $\pm$ 1.07
0.25	37.61 $\pm$ 0.95	47.49 $\pm$ 1.41	51.44 $\pm$ 0.82	54.57 $\pm$ 0.37

Table 1: Baseline proof performance at different temperatures. Results are reported as mean $\pm$ standard deviation across five seeds on 243 MiniF2F test problems. Bold values indicate the best observed temperature for each model and value of k .

Temperature τ	Pass@1 cost	Pass@4 cost	Pass@8 cost	Pass@16 cost
Apertus-8B-finetuned				
1.00	480 $\pm$ 21	1 920 $\pm$ 31	3 925 $\pm$ 72	7 781 $\pm$ 92
0.50	524 $\pm$ 49	1 893 $\pm$ 57	3 848 $\pm$ 43	7 773 $\pm$ 72
0.25	593 $\pm$ 28	2 367 $\pm$ 57	4 663 $\pm$ 54	9 363 $\pm$ 120
DeepSeek-Prover-v2-cpt-sft-1e				
1.00	1 027 $\pm$ 25	4 115 $\pm$ 46	8 314 $\pm$ 33	16 581 $\pm$ 137
0.50	1 100 $\pm$ 20	4 415 $\pm$ 54	8 893 $\pm$ 64	17 683 $\pm$ 184
0.25	1 139 $\pm$ 21	4 681 $\pm$ 21	9 408 $\pm$ 67	18 715 $\pm$ 141

Table 2: Generated-token cost for the baseline sampling experiments. Costs are reported as generated tokens per theorem and include all generated proof candidates. Values are reported as mean $\pm$ standard deviation across five seeds.

5.2 Power sampling

5.2.1 Metropolis Hastings power sampling

Table 3 shows the results from the Metropolis–Hastings power-sampling experiment. These results are from runs on the MiniF2F validation set and are therefore not directly comparable with the test-set results reported for the other methods. The sampler reaches 47.12% validation accuracy, which is roughly comparable to the cheap Scalable Power Sampling configuration on the same split.

The main limitation of this method is its cost. The sampler uses more than 52,000 generated tokens per theorem and takes roughly 75 hours per seed. Because of this high runtime and token cost, we did not include the MCMC sampler in the full test-set experiments and instead focus the remaining evaluation on Scalable Power Sampling and adaptive controllers.

Metric	Mean	Std
Accuracy (%)	47.12	0.62
Generated tokens / theorem	52 412	558
Runtime / seed (hours)	74.8	1.4

Table 3: Metropolis–Hastings block-resampling results on MiniF2F validation using Apertus-8B-finetuned. Results are reported over two seeds. The method was evaluated only on the validation set due to its high runtime.

5.2.2 Scalable Power Sampling

We next look into the results from Scalable Power Sampling on the MiniF2F test set. We first consider the base sampling modes for Apertus-8B-finetuned at pass@1, shown in Table 4. The vanilla row corresponds to ordinary sampling without rollouts, while the cheap and expensive modes use the budgets described in Section 4.4.3.

The results show a clear jump in accuracy when moving from the vanilla mode to the cheap mode. Vanilla reaches 32.76%, while the cheap mode reaches around 40%. However, increasing the budget further from cheap to expensive does not show a clear additional gain. The expensive configurations achieve accuracies between 38.85% and 40.82%, which is similar to the cheap configs despite using substantially more compute. We also do not see any large difference between the evaluated values of the sharpening parameter α . Within each budget mode, the differences between $\alpha = 1, 2, 4$ are small and correspond to only a few solved problems.

The cost differences are however much larger than the accuracy differences. The cheap mode increases generated-token cost from about 542 tokens per theorem for vanilla to around 2,000 tokens per theorem. The expensive mode increases this further to over 7,000 tokens per theorem. Thus, although power sampling improves over vanilla pass@1 sampling, the more expensive rollout budget does not produce a proportional accuracy improvement in this setting.

Mode	α	Pass@1 (%)	Generated tokens / theorem
Vanilla	1	32.76 ± 2.61	542 ± 36
Cheap	1	40.33 ± 0.82	$2\,128 \pm 89$
Cheap	2	39.51 ± 1.10	$2\,028 \pm 150$
Cheap	4	40.00 ± 0.80	$2\,052 \pm 118$
Expensive	1	38.85 ± 1.15	$7\,200 \pm 414$
Expensive	2	40.82 ± 1.69	$7\,509 \pm 345$
Expensive	4	40.82 ± 0.66	$7\,262 \pm 201$

Table 4: Scalable power-sampling results on the MiniF2F test set using Apertus-8B-finetuned. Results are reported as mean $\pm$ standard deviation over five seeds. Generated token counts are reported per theorem.

In addition, we evaluate the cheap Scalable Power Sampling configuration at pass@4, shown in Table 5. The jump in accuracy seen at pass@1 does not seem to continue here. At pass@1, the cheap power-sampling mode improves over the Apertus baseline pass@1 results by roughly 1–6 percentage points, depending on the baseline temperature. At pass@4, however, the cheap power-sampling runs reach around 45.3%–45.5%, which is essentially the same as the strongest Apertus baseline pass@4 result of 45.51%.

The cost differences are however much larger than the accuracy differences. The cheap power-sampling pass@4 runs use around 8,400 generated tokens per theorem, compared with 1,920 generated tokens per theorem for the temperature-1.0 baseline at pass@4. Thus, the cheap power-sampling settings do not seem to give a clear advantage over simply drawing more independent samples of the baseline mode.

α	Pass@4 (%)	Generated tokens / theorem
1	45.43 ± 0.85	$8\,355 \pm 232$
2	45.51 ± 0.67	$8\,415 \pm 176$
4	45.27 ± 0.86	$8\,527 \pm 168$

Table 5: Pass@4 results for the cheap Scalable Power Sampling configuration on the MiniF2F test set using Apertus-8B-finetuned. Results are reported as mean $\pm$ std over five seeds. Generated token counts are reported per theorem.

We also evaluate the cheap Scalable Power Sampling configuration on DeepSeek-Prover-v2-cpt-sft-1e, shown in Table 6. Compared with the DeepSeek baseline pass@1 results, cheap power sampling improves over the temperature-1.0 baseline of 33.50%, and is also above the lower-temperature baselines of 36.46% and 37.61%. The best cheap power-sampling result is 40.25% at $\alpha = 4$, but the differences between values of α are again small relative to seed variation.

The token cost is substantially higher than for ordinary generation. The cheap power-sampling runs use roughly 5,000 generated tokens per theorem, compared with 1,027–1,139 generated tokens per theorem for the DeepSeek pass@1 baselines. Thus, as with Apertus, Scalable Power Sampling gives a pass@1 improvement over ordinary sampling, but the improvement comes at a large token cost and does not clearly depend on α .

Model	α	Pass@1 (%)	Generated tokens / theorem
DeepSeek-Prover	1	39.42 ± 1.20	$5\,049 \pm 137$
DeepSeek-Prover	2	38.44 ± 0.85	$5\,283 \pm 254$
DeepSeek-Prover	4	40.25 ± 1.65	$5\,109 \pm 124$

Table 6: Scalable power-sampling results for DeepSeek-Prover-v2-cpt-sft-1e on the MiniF2F test set. All runs use the cheap two-phase sampling configuration and report Pass@1 results. Values are reported as mean $\pm$ standard deviation over five seeds. Generated token counts are reported per theorem and include all generated proof attempts.

5.3 Adaptive controllers

5.3.1 Phased controllers

We next look into the results from the phased controllers described in Section 3.5.1. Table 7 reports both pass@1 and pass@4 results for the three phased schedules selected from validation tuning.

At pass@1, the phased controllers achieve accuracies between 40.41% and 41.23%. This is comparable to the best static Scalable Power Sampling configurations, but does not show a clear improvement over them. Among the three variants, Phased v2 performs best, reaching 41.23%, but the differences between the different versions are very small relative to the seed-to-seed variation. The generated-token costs are between the cheap and expensive static modes, ranging from roughly 3,500 to 3,900 tokens per theorem.

At pass@4, the phased controllers reach around 44.9%–45.2%. This is again close to the strongest Apertus baseline pass@4 result of 45.51%, but at a much higher generated-token cost. The phased pass@4 runs use roughly 13,800–15,500 generated tokens per theorem, compared with 1,920 tokens per theorem for the temperature-1.0 baseline.

Controller	r_{exp}	r_{vanilla}	Pass@1		Pass@4	
			Acc. (%)	Tokens / theorem	Acc. (%)	Tokens / theorem
Phased v1	0.05	0.75	40.41 $\pm$ 1.41	3 495 $\pm$ 78	45.02 $\pm$ 0.42	14 243 $\pm$ 133
Phased v2	0.05	0.60	41.23 $\pm$ 0.60	3 482 $\pm$ 81	45.19 $\pm$ 0.48	13 817 $\pm$ 285
Phased v3	0.10	0.50	40.49 $\pm$ 1.62	3 921 $\pm$ 66	44.86 $\pm$ 0.52	15 519 $\pm$ 115

Table 7: Results for the heuristic phased controllers on the MiniF2F test set using Apertus-8B-finetuned. All runs use power sampling with $\alpha = 2$. Results are reported as mean $\pm$ std over five seeds. Here r_{exp} is the ratio until where the expensive mode is used, while r_{vanilla} is the ratio after which the controller switches to vanilla mode. Generated token counts are averaged per theorem and include all generated proof attempts.

5.3.2 Learned Controllers

Table 8 shows the results for the learned XGBoost controller. We evaluate three values of the cost weight λ , which controls the trade-off between proof success and cost.

At pass@1, the learned controller reaches results between 39.26% and 40.49%. This is comparable to the static cheap Scalable Power Sampling configurations, but does not give a clear improvement over them. The best pass@1 result is obtained with $\lambda = 0$, where the controller ignores the predicted cost term and selects actions only according to predicted success. Increasing the cost weight slightly reduces generated-token cost, but the effect is small and does not produce a clear accuracy–cost trade-off.

At pass@4, the learned controller reaches around 45.3%–45.4%. This is again close to the strongest Apertus baseline pass@4 result of 45.51%, but also again requires a lot more generated tokens. The XGBoost pass@4 runs use roughly 7,800–8,600 generated tokens per theorem, compared with 1,920 for the temperature-1.0 baseline. Overall, the learned controller seems to allocate compute similarly to cheap power sampling, but it does not seem to give a meaningful improvement over independent pass@4 sampling.

Cost weight λ	Pass@1		Pass@4	
	Acc. (%)	Tokens / theorem	Acc. (%)	Tokens / theorem
0	40.49 $\pm$ 1.24	2 009 $\pm$ 86	45.35 $\pm$ 0.48	8 576 $\pm$ 183
0.005	39.26 $\pm$ 0.85	1 955 $\pm$ 126	45.27 $\pm$ 0.34 [†]	7 838 $\pm$ 357 [†]
0.05	39.84 $\pm$ 0.92	1 998 $\pm$ 143	45.43 $\pm$ 0.20	8 065 $\pm$ 249

Table 8: Results for the learned XGBoost controller on the MiniF2F test set using Apertus-8B-finetuned. Results are reported as mean $\pm$ std over five seeds, except entries marked with [†]. Generated token counts are averaged per theorem. The $\lambda = 0.005$ pass@4 entry is computed over three seeds because two runs were contaminated by stale result files.

To better understand the learned policy, we also inspect the action choices made in one evaluation run. At the

step level, the controller selects the cheap mode for approximately 70% of decisions, vanilla for 27%, and the expensive mode for only 3%. This supports the interpretation that the learned controller mostly behaves like a low-cost power-sampling policy, rather than frequently allocating larger rollout budgets to selected proof states.

5.4 Additional experiments

5.4.1 Feedback extension

Table 9 reports the results of the feedback-model experiments. The feedback baseline reaches 42.80% pass@1 accuracy, which is higher than the no-feedback DeepSeek-Prover pass@1 baselines reported above. These runs use a longer maximum generation length than the main baseline experiments in order to accommodate the additional tokens needed for the feedback blocks.

We also evaluate the feedback model with cheap Scalable Power Sampling using $\alpha = 2$. This reaches 42.94% accuracy, which is essentially unchanged from the feedback baseline. The generated-token cost, however, increases substantially, from around 4,300 to about 24,300 tokens per theorem. Thus, in this setting, adding power sampling to the feedback model does not provide a meaningful accuracy improvement, but increases token cost by more than a factor of five.

Method	Seeds	Accuracy (%)	Generated tokens / theorem
Feedback	3	42.80 $\pm$ 1.65	4 347
Feedback + cheap SPS, $\alpha = 2$	3	42.94 $\pm$ 1.85	24 254

Table 9: Results on the MiniF2F test set using DeepSeek-Prover-v2-cpt-sft-feedback-1e. Results are reported as mean $\pm$ standard deviation over three seeds. Generated token counts are averaged per theorem.

5.4.2 Uncertainty analysis

Error localization Figure 1 shows that token-level uncertainty is often aligned with the first error reported by Lean. In the left plot, the CDF lies far above the uniform baseline, indicating that the token aligned with the first Lean error is often among the most uncertain tokens in the proof body. In particular, the error-aligned token is in the top 5% most uncertain proof tokens in 48.5% of proofs, in the top 10% in 65.4%, and in the top 20% in 76.9%. This suggests that model uncertainty provides a useful localization signal for verifier failures.

The right plot compares the position of the first reported Lean error with the position of the highest-uncertainty token in the proof body. The distance is normalized by proof-body length, and negative values mean that the highest-uncertainty token occurs before the reported Lean error. The highest-uncertainty token occurs before or at the first Lean error in 53.8% of proofs, and the histogram has a large spike at normalized distance 0. This supports the view that high-uncertainty regions often occur near the point where Lean first reports a failure. However, the spread of the distribution also shows that uncertainty is not a precise error locator.

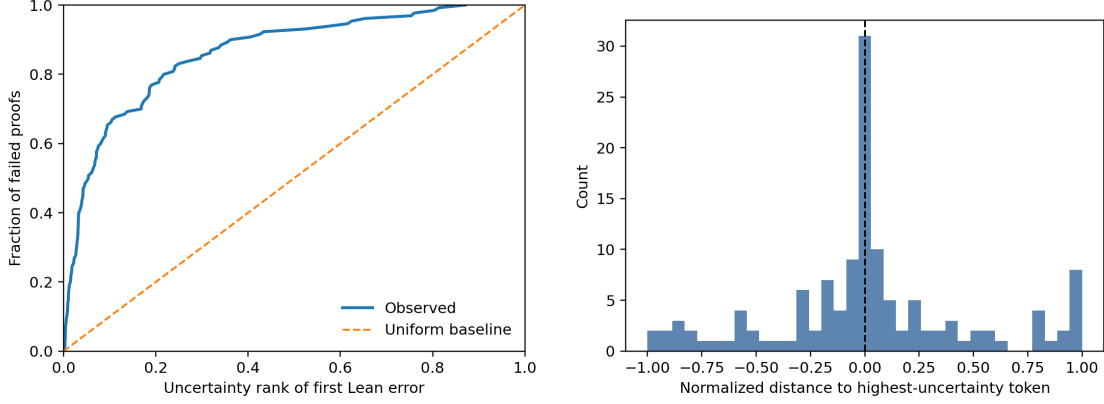


Figure 1: Token-level uncertainty and first Lean errors on failed MiniF2F validation proofs. Left: CDF of the uncertainty-rank percentile of the token aligned with the first Lean error. The dashed line is the uniform baseline. Right: normalized distance between the highest-uncertainty token and the token aligned with the first Lean error, where negative values mean that the highest-uncertainty token occurs before the Lean error.

Classifying proof candidates We next evaluate whether aggregate uncertainty over a full generated completion is predictive of Lean verification success. The results show a clear signal. Among the 235 candidates that reached Lean verification, 103 passed and 132 failed. Ranking candidates by lower total generated NLL gives an AUC of 0.822, substantially above the random baseline of 0.5. The median total NLL is also much lower for successful candidates than for failed candidates, 5.31 compared with 13.78. Length-normalized and local uncertainty metrics show the same trend, with mean token NLL giving an AUC of 0.725, and maximum token NLL giving an AUC of 0.743. Successful candidates have lower median mean NLL (0.015 versus 0.040) and lower median maximum-token NLL (2.24 versus 3.42).

These results suggest that the model’s likelihood estimates contain useful information about proof correctness. However, the experiment does not isolate candidate-level correctness from theorem difficulty or proof length. Harder theorems may naturally produce both higher-uncertainty completions and lower verification rates. We therefore treat this as evidence that uncertainty is predictive of Lean success, rather than as a direct demonstration that uncertainty can select the best proof among multiple candidates for the same theorem.

Uncertainty metric	AUC	Pass median	Fail median
Total generated NLL	0.822	5.31	13.78
Mean generated NLL	0.725	0.015	0.040
Max token NLL	0.743	2.24	3.42

Table 10: Uncertainty per proof candidate as a predictor of Lean verification success on MiniF2F validation.

5.4.3 Results on other benchmarks

Finally, we run a small set of preliminary experiments on the more difficult benchmarks ProofNet and MoBench. We evaluate Apertus-8B-finetuned with ordinary pass@1 sampling, cheap Scalable Power Sampling with $\alpha = 2$, and the learned XGBoost controller. These experiments are intended as lightweight robustness checks, and not full benchmark evaluations, since the power-sampling and XGBoost runs were completed for only one seed.

Dataset	Method	Accuracy (%)	Generated tokens / theorem
ProofNet	Baseline pass@1	1.29 ± 0.73	877 ± 13
ProofNet	Cheap SPS, $\alpha = 2$	3.23	4 109
ProofNet	XGBoost	2.15	3 739
MoBench	Baseline pass@1	0.44 ± 0.38	789 ± 15
MoBench	Cheap SPS, $\alpha = 2$	0.56	3 242
MoBench	XGBoost	1.11	3 302

Table 11: Preliminary pass@1 results on ProofNet and MoBench using Apertus-8B-finetuned. Baseline results are reported as mean $\pm$ standard deviation over five seeds. Cheap SPS and XGBoost results are reported for one seed only. Generated-token counts are averaged per theorem.

The accuracies are much lower than on MiniF2F. On ProofNet, the baseline solves only 1.29% of problems on average, while cheap SPS and XGBoost solve 3.23% and 2.15%, respectively, in the single completed seed. On MoBench, all methods remain close to zero accuracy. Although the one-seed SPS and XGBoost runs sometimes solve a few more problems than the baseline mean, the absolute number of solved problems is very small and the generated-token cost increases by roughly a factor of four. These results should therefore be interpreted cautiously, as the observed differences could be due to seed variability, and more runs would be needed before drawing conclusions.

6 Discussion

This section discusses the main findings from the experiments. The overall goal of the project was to investigate whether inference-time compute can be used more effectively when generating Lean proofs. We focus on what the results suggest about distribution sharpening, accuracy–cost trade-offs, adaptive controllers, and uncertainty as a signal for proof generation.

6.1 Effect of distribution sharpening

The results suggest that there is some benefit to allocating additional compute during generation. For both Apertus and DeepSeek-Prover, the cheap Scalable Power Sampling configuration improves pass@1 performance compared with the vanilla budget or ordinary sampling baselines. This indicates that rollout-based evaluation can help the model select better proof continuations, at least when we look at a single proof attempt in isolation.

The results do however not show a clear effect of the power exponent α . Across the configurations we evaluated, changing α from 1 to 2 or 4 only changes the final accuracy by a small number of solved problems, and these differences are often comparable to the variation across the seeds. This suggests that the improvement may not come directly from distribution sharpening. Instead, part of the gain may come from giving the model additional opportunities to generate and compare candidate chunks before committing to a continuation.

There are several possible explanations for this. One possible reason is that the rollout budgets used in our experiments were too small to create a strong sharpening effect. In that case, larger rollout budgets or longer horizons might make the effect of α more visible. Another possibility is that, in this case, the main benefit of the method comes from local candidate selection instead of from approximating the full sequence-level power distribution.

6.2 Accuracy–cost trade-offs at pass@1 and pass@k

One important result of the project is that higher single-attempt accuracy does not necessarily imply a better accuracy–cost trade-off. The cheap Scalable Power Sampling configurations improve pass@1 performance compared with ordinary sampling, but this improvement comes at a substantially higher token cost. Increasing the rollout budget further does not seem to give a proportional gain, suggesting that the benefit of additional per-attempt compute quickly saturates in this setting.

The pass@k results make this trade-off even clearer. When we evaluate several proof attempts, ordinary independent sampling is highly competitive. Here, spending compute on generating multiple diverse candidates can be more effective than spending the same compute on making each individual candidate stronger. This is

especially relevant for theorem proving, where different proof attempts may explore different tactics, lemmas, or proof structures. A sharper method may improve the average quality of a single trajectory, but it can also reduce diversity between attempts. At the current accuracy–cost trade-off, for the power-sampling methods to be actually helpful they would have to unlock the ability to solve a new set of proofs, which the base models are too weak to solve. In our experiments, neither the static nor adaptive power-sampling methods showed such an effect clearly.

Overall, these results suggest that additional inference-time compute is useful, but only when allocated in a way that gives either better single-attempt quality or useful diversity. In our experiments, the first increase from ordinary sampling to cheap rollout-based candidate selection improves pass@1, while further increases in per-attempt compute show diminishing returns. For larger candidate budgets, strong independent sampling is a difficult baseline to beat.

6.3 Takeaways from adaptive controllers

The adaptive controller experiments did not show a clear improvement over the strongest fixed-budget configurations. The phased controllers increased generated-token cost relative to the cheap static mode, but did not produce a meaningful increase in accuracy. This suggests that simply allocating more compute early in the proof and less compute later is not sufficient, at least for the phase schedules tested here.

One important issue is that the available sampling modes were not strongly separated in the final test results. In the static experiments, the cheap and expensive modes achieved similar accuracies, with the main difference being token cost. This makes adaptive control difficult. If the expensive mode does not reliably solve more problems than the cheap mode, then a controller has little opportunity to improve the accuracy–cost trade-off by switching between them. This was somewhat different during validation, where we observed larger differences between budget modes and values of α , which motivated the final mode choices. However, these differences did not transfer clearly to the test set.

The learned XGBoost controller also did not clearly improve the accuracy–cost trade-off. Its pass@1 results are comparable to the cheap Scalable Power Sampling configuration, and its token cost remains around the same range. Inspecting the selected actions shows that the controller chooses the cheap mode for most decisions, uses vanilla generation for a smaller fraction, and rarely selects the expensive mode. This behaviour is understandable, since the cheap mode already appears to have the best trade-off among the available fixed configurations. However, it also means that the learned controller mostly recovers a low-cost static policy rather than discovering a substantially better adaptive allocation strategy. This again might have been different if the differences between the different modes were larger.

These results do not rule out adaptive compute allocation, but they highlight several requirements for it to be effective. First, the action space must contain modes with meaningfully different accuracy–cost behaviour, not only different costs. Second, the controller needs features that can predict when additional compute is actually useful. Third, the training signal is sparse: the XGBoost controller is trained from final Lean outcomes, so every intermediate decision in a proof receives the same success label. This makes it difficult to identify which particular decisions were responsible for success or failure. A stronger adaptive method may therefore require more informative budget modes, richer proof-state features, counterfactual training data, or labels that better connect intermediate decisions to final verification outcomes.

6.4 Uncertainty as a signal for proof generation

The uncertainty experiments suggest that model likelihoods contain useful information about proof failures. Both the localization and candidate-level results indicate that high uncertainty is correlated with fragile or unsuccessful proof attempts. This makes uncertainty a promising diagnostic signal for proof generation, especially because it can be computed directly from the model without requiring additional verifier calls.

However, uncertainty should of course not be interpreted as a direct measure of correctness. The first error reported by Lean is not necessarily the location of the first mathematical mistake. A proof may take a wrong direction several lines before Lean reports an error, or fail because of a local syntax or tactic issue that is not representative of the deeper proof structure. Similarly, for full proof candidates, uncertainty may partly reflect theorem difficulty or proof length rather than only whether a specific generated proof is correct.

There are also cases where uncertainty can be misleading. A model can be confidently wrong if it generates a

common but invalid tactic pattern. On the other hand, a correct proof may require an unusual lemma, theorem name, or proof step that receives low model probability. High uncertainty may therefore indicate either an error-prone region or a genuinely difficult but necessary part of the proof.

The most useful interpretation is therefore that uncertainty is an informative but imperfect signal. It may help decide when to allocate more samples to a theorem, which proof attempts to prioritize for verification, or which regions to target for repair or resampling. This also suggests a possible direction for future adaptive methods. Instead of using uncertainty as a standalone correctness predictor, it could be combined with proof-state features, verifier feedback, or candidate diversity measures to guide compute allocation more effectively.

6.5 Limitations

The experiments we performed have several limitations. First, the main evaluation is performed on MiniF2F, which is a standard benchmark but not a uniform collection of problems. Some theorems are solved reliably by ordinary sampling, while others appear to be much harder for the models used in this project. This can make it difficult to obtain large additional gains from inference-time sampling alone. Once the easier and near-threshold problems have been solved, the remaining unsolved problems may require different proof ideas, stronger models, or more structured search rather than only a sharper sampling distribution.

A related limitation is that several method choices were tuned on the validation set, where we observed larger differences between budget modes and values of α than in the final test results. This motivated the fixed modes and controller action spaces used in the main experiments. However, these differences did not transfer clearly to the test set, where many configurations achieved similar accuracies and mainly differed in token cost.

Second, the compute budgets explored in this project are limited. For both Metropolis–Hastings power sampling and Scalable Power Sampling, larger budgets might produce a closer approximation to the intended power distribution. More refinement steps, longer rollout horizons, or larger candidate sets could make the effect of distribution sharpening more visible. However, such runs would be substantially more expensive and time-consuming. The results therefore reflect the practical budget regime explored in this project, not necessarily the full potential of the methods under much larger inference-time compute.

Third, the adaptive controllers are limited by their action space, features, and training signal. The heuristic controllers use only proof position, while the learned controller is trained from sparse final Lean outcomes. This makes it difficult to identify which generation steps actually benefit from additional compute. In addition, the training data is not fully counterfactual. For most proof states, we only observe the outcome of the action that was actually taken, not what would have happened if a different sampling mode had been chosen at the same state. The controller may therefore learn mostly problem difficulty or general mode preferences, rather than a precise action-specific value function.

7 Conclusion

In this project, we investigated whether inference-time compute can be allocated more effectively for Lean proof generation. We evaluated ordinary independent sampling, Metropolis–Hastings power sampling, Scalable Power Sampling, simple phase-based controllers, a learned XGBoost controller, in addition to investigating uncertainty-based diagnostics. The main goal was to understand whether additional compute can improve proof success, and whether this compute is better spent on strengthening individual proof attempts or on generating more diverse candidates.

The experiments show that additional compute can improve single-attempt proof generation. In particular, cheap Scalable Power Sampling improved pass@1 performance compared with ordinary sampling baselines. However, these gains came at a substantially higher generated-token cost, and the effect of the power exponent α was not clear in the final test results. At larger candidate budgets, ordinary independent sampling remained highly competitive, suggesting that diversity across proof attempts is often more cost-effective than spending substantially more compute on each individual attempt. In the current setup, the power-sampling methods did not seem to unlock a separate set of problems that could not already be reached efficiently through independent sampling.

The adaptive controllers explored in this project did not clearly improve over the best fixed-budget configurations. The phase-based controllers increased token cost without producing a meaningful accuracy gain, and

the learned XGBoost controller mostly recovered a low-cost policy similar to cheap Scalable Power Sampling. These results suggest that adaptive compute allocation is challenging when the available sampling modes do not differ clearly in accuracy, or when the controller is trained from sparse final verification labels.

Finally, the uncertainty experiments suggest that model likelihoods contain useful information about proof failures. Token-level uncertainty is often related to regions where Lean reports errors, and aggregate uncertainty over a proof attempt is predictive of verification success. This makes uncertainty a promising diagnostic signal, although not a standalone correctness measure. Overall, the project shows that inference-time compute is useful for Lean proof generation, but also that simply spending more compute per attempt is not enough. Effective methods must balance single-attempt quality, diversity across attempts, and generated-token cost.

7.1 Future Work

One important direction for future work is to improve the adaptive controllers. The learned controller in this project was trained from sparse final verification labels, and the available sampling modes were not clearly separated in the final test results. Future work could collect more informative training data by branching from the same proof state and trying several sampling modes. This would provide more counterfactual information about which actions are actually useful in a given state. It may also be useful to define a richer action space, where the controller can adjust parameters such as rollout horizon, number of candidates, or number of rollouts more flexibly.

The uncertainty results also suggest a promising direction for adaptive proof generation. Instead of using a fixed $\text{pass@}k$ budget for every theorem, future methods could allocate more attempts to theorems or candidates with high uncertainty, while stopping earlier when the model appears confident. Uncertainty could also be used to select regions of a failed proof for repair or resampling. Combining uncertainty with proof-state information, verifier feedback, or candidate diversity measures may provide a stronger signal for deciding where additional inference-time compute is most useful.

Finally, future work should evaluate these methods on a broader set of benchmarks. Most experiments in this project were performed on MiniF2F, and the exploratory runs on harder benchmarks were limited. More complete evaluations on diverse Lean benchmarks would help determine whether the observed trends are specific to MiniF2F, or whether they hold across problems with different difficulty levels and proof styles.

Acknowledgements

I am grateful to my supervisor, Etienne Bamas, for giving me the opportunity to work on this project, and for his guidance and feedback throughout the process. I would also like to thank the RCP cluster team for providing access to the computational resources used in the experiments.

Statement on the use of AI tools

Generative AI tools were used as supportive tools during this project. They were used to assist with code debugging, command-line troubleshooting, LaTeX formatting, and language polishing. They were also used as a discussion partner when refining explanations and improving the clarity of the report.

All experiments, evaluations, and analyses reported in this project were carried out and reviewed by the author. The generated results, tables, figures, and interpretations were checked against the underlying experiment outputs. The final implementation choices, conclusions, and written report remain the responsibility of the author.

References

- [1] Zhangir Azerbayev, Bartosz Piotrowski, Hailey Schoelkopf, Edward W. Ayers, Dragomir Radev, and Jeremy Avigad. Proofnet: Autoformalizing and formally proving undergraduate-level mathematics, 2023.
- [2] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901, 2020.
- [3] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code, 2021.
- [4] Tianqi Chen and Carlos Guestrin. Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '16*, page 785–794. ACM, August 2016.
- [5] Leonardo de Moura and Sebastian Ullrich. The lean 4 theorem prover and programming language. In *Automated Deduction – CADE 28*, pages 625–635. Springer, 2021.
- [6] Tom Fawcett. An introduction to roc analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006.
- [7] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration, 2020.
- [8] Thomas Hubert, Rahul Mehta, Laurent Sartran, et al. Olympiad-level formal mathematical reasoning with reinforcement learning. *Nature*, 651:607–613, 2026.
- [9] Xiaotong Ji, Rasul Tutunov, Matthieu Zimmer, and Haitham Bou Ammar. Scalable power sampling: Unlocking efficient, training-free reasoning for llms via distribution sharpening, 2026.
- [10] Aayush Karan and Yilun Du. Reasoning with sampling: Your base model is smarter than you think. *arXiv*, 2025.
- [11] Yong Lin, Shange Tang, Bohan Lyu, Ziran Yang, Jui-Hui Chung, Haoyu Zhao, Lai Jiang, Yihan Geng, Jiawei Ge, Jingruo Sun, Jiayun Wu, Jiri Gesi, Ximing Lu, David Acuna, Kaiyu Yang, Hongzhou Lin, Yejin Choi, Danqi Chen, Sanjeev Arora, and Chi Jin. Goedel-prover-v2: Scaling formal theorem proving with scaffolded data synthesis and self-correction, 2025.
- [12] Project Apertus, Alejandro Hernández-Cano, Alexander Hägele, Allen Hao Huang, Angelika Romanou, et al. Apertus: Democratizing open and compliant LLMs for global language environments, 2025.
- [13] Z. Z. Ren, Zhihong Shao, Junxiao Song, Huajian Xin, Haocheng Wang, Wanbiao Zhao, Liyue Zhang, Zhe Fu, Qihao Zhu, Dejian Yang, Z. F. Wu, Zhibin Gou, Shirong Ma, Hongxuan Tang, Yuxuan Liu, Wenjun Gao, Daya Guo, and Chong Ruan. Deepseek-prover-v2: Advancing formal mathematical reasoning via reinforcement learning for subgoal decomposition, 2025.
- [14] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024.
- [15] George Tsoukalas, Jasper Lee, John Jennings, Jimmy Xin, Michelle Ding, Michael Jennings, Amitayush Thakur, and Swarat Chaudhuri. Putnambench: Evaluating neural theorem-provers on the putnam mathematical competition, 2024.
- [16] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model?, 2025.
- [17] Tunyu Zhang, Haizhou Shi, Yibin Wang, Hengyi Wang, Xiaoxiao He, Zhuowei Li, Haoxian Chen, Ligong Han, Kai Xu, Huan Zhang, Dimitris Metaxas, and Hao Wang. Tokur: Token-level uncertainty estimation for large language model reasoning, 2026.
- [18] Kunhao Zheng, Jesse Michael Han, and Stanislas Polu. Minif2f: a cross-system benchmark for formal olympiad-level mathematics, 2022.

Appendix A Pseudocode for Scalable Power Sampling

Algorithm 1: Block-level Scalable Power Sampling for Lean proof generation

Input: Prompt q , model p_θ , exponent α , maximum length T
Input: Chunk size c , candidate budget B , evaluated candidates K
Input: Rollouts per candidate R , rollout horizon H
Output: Generated Lean proof continuation x

```

 $x \leftarrow \emptyset$ ;
while  $|x| < T$  and no stop sequence has been generated do
  // Generate candidate next chunks
  for  $i \leftarrow 1$  to  $B$  do
    Sample candidate chunk  $z_i \sim p_\theta(\cdot \mid q, x)$  with length at most  $c$ ;
     $\ell_i \leftarrow \log p_\theta(z_i \mid q, x)$ ;
  end
   $\mathcal{I} \leftarrow$  indices of the  $K$  candidates with largest  $\ell_i$ ;
  // Estimate future scaling terms
  foreach  $i \in \mathcal{I}$  do
    for  $r \leftarrow 1$  to  $R$  do
      Sample rollout  $y_{i,r} \sim p_\theta(\cdot \mid q, x, z_i)$  with length at most  $H$ ;
       $m_{i,r} \leftarrow \log p_\theta(y_{i,r} \mid q, x, z_i)$ ;
    end
     $\widehat{\xi}(z_i) \leftarrow \frac{1}{R} \sum_{r=1}^R \exp((\alpha - 1)m_{i,r})$ ;
  end
  // Approximate the block-level power distribution
  foreach  $i \in \mathcal{I}$  do
     $s_i \leftarrow \exp(\alpha \ell_i) \widehat{\xi}(z_i)$ ;
  end
   $Z \leftarrow \sum_{j \in \mathcal{I}} s_j$ ;
  Sample  $i^* \in \mathcal{I}$  with probability  $s_i/Z$ ;
   $x \leftarrow x \parallel z_{i^*}$ ;
end
Apply deterministic Lean extraction and cleanup to  $x$ ; return  $x$ ;

```

Appendix B Pseudocode for Metropolis–Hastings Power Sampling

Algorithm 2: Block-level Metropolis–Hastings power sampling for Lean proof generation

Input: Prompt q , base model p_θ , proposal model q_φ , exponent α
Input: Maximum length T , block size c , MCMC steps per block M
Output: Generated Lean proof continuation x

$x \leftarrow \emptyset$;
while $|x| < T$ *and no stop sequence has been generated* **do**
 // Initialize next block by autoregressive extension
 Sample extension $z \sim q_\varphi(\cdot \mid q, x)$ with length at most c ;
 $x_{\text{cur}} \leftarrow x \parallel z$;
 // Run Metropolis–Hastings refinement within the current block
 for $m \leftarrow 1$ **to** M **do**
 Sample cut position r uniformly from allowed proof-body positions in x_{cur} ;
 Let a be the prefix of x_{cur} before position r ;
 Sample replacement suffix $b' \sim q_\varphi(\cdot \mid q, a)$ up to the current block boundary;
 $x_{\text{prop}} \leftarrow a \parallel b'$;
 Compute

$$A = \min \left(1, \frac{p_\theta(x_{\text{prop}} \mid q)^\alpha q_\varphi(x_{\text{cur}} \mid x_{\text{prop}}, q)}{p_\theta(x_{\text{cur}} \mid q)^\alpha q_\varphi(x_{\text{prop}} \mid x_{\text{cur}}, q)} \right)$$

 ;

 Draw $u \sim \text{Uniform}(0, 1)$;
 if $u < A$ **then**
 $x_{\text{cur}} \leftarrow x_{\text{prop}}$;
 end
 end
 // Fix the refined block and continue
 $x \leftarrow x_{\text{cur}}$;
end
Apply deterministic Lean extraction and cleanup to x ; **return** x ;